

```
(:number person))]]]
      (update-person connection edited-person)))
      (print "could not find a person with that id"))
      True))
```

Compared to editing, removing a person is a rather simple operation. The code asks for the ID of the person to be removed and calls an appropriate function to perform the removal:

```
(defn remove-person [connection]
  (print "*****")
  (print " delete person")
  (print "")
  (let [[person-id (key-input "enter id of person to delete:
")]
    (delete-person connection person-id))
  True)
```

The function for ending the program closes the database connection and returns 'False'. This causes the main loop to exit and the program to terminate:

```
(defn quit [connection]
  (.close connection)
  False)
```

Tying it together in the main menu

Finally, add the following code to create a main menu. Here, we are using a dictionary to map the numbers a user may enter to functions that should be executed. In case the user enters something that we don't understand, we provide short instructions on how to operate:

```
(defn main-menu [connection]
  (let [[menu-choices ("1" add-person
                        "2" search-person
                        "3" edit-person
                        "4" remove-person
                        "5" quit)]]
    (print "*****")
    (print " register")
    (print "")
    (print "1. add new person")
    (print "2. search")
    (print "3. edit person")
    (print "4. delete person")
    (print "5. quit")
    (print "")
    (try (let [[selection (get menu-choices (key-input "make
a selection: ")]])
          (selection connection))
        (catch [e KeyError] (print "Please choose between 1 and
5") True))))
```

```
(if (= __name__ "__main__")
  (let [[connection (create-schema (get-connection))]]
    (while (main-menu connection) [])))
```

Here we are checking the `__name__` attribute and if it is `'__main__'`, we know that the module has been started from the command line as a program and not included as a part of some other program. If this is the case, we call the functions we defined earlier in order to create our database and display the main menu. The `'while'` loop keeps running as long as the conditional evaluates to `'true'`. And since the value of the conditional is the same as the value returned by our functions, the program finishes as the quit function returns `'false'`.

You can try out the program at this point by issuing the following command in the command line:

```
hy register.hy
```

As a result, there should be a new file in the folder called `'register.db'` and the main menu is shown in the terminal. You can now add, search, edit and remove people and their phone numbers in your personal phonebook program.

What next

Hy is a young language and thus changing. It is possible to use existing Lisp tutorials, but not every feature mentioned in them is currently available in Hy, and some of the features might work slightly differently. However, the basic idea behind the language is still the same. And you already have a working phonebook program at your disposal. Why not try and add a new field for storing addresses or sort the search results by the name of the person, instead of the ID number.

If you are using emacs for writing programs, there is Hy mode available for it at: <https://github.com/hylang/hy-mode>. Only emacs24 and newer are supported, as of writing this article.



Note:

- Since Hy is a relatively young language, it is evolving very fast. This article assumes that the version used is 0.9.10.
- Example code in this article is available as a zip archive at <https://github.com/turtto/register/zipball/master>.
- For more information about Hy, take a look at <http://docs.hylang.org/en/latest/>.
- Hy, like Python, has an interactive interpreter. You can start it by typing Hy in the terminal and close it by pressing Ctrl+D.
- Developers of Hy and people interested in the language collaborate in [hy](https://www.hycon.org/) on [irc.freenode.net](https://www.hycon.org/).



By: Tuukka Turto

The author is a software aficionado.